

Asterisk CTI 실전 개발 산출물

DB ERD · PostgreSQL SQL 스키마 · REST/WebSocket API 명세 · Asterisk 설정 파일 초안

오픈소스 IP-PBX Asterisk 기반 콜센터 CTI 구축 프로젝트

본 문서는 업로드된 실전 개발용 상세 설계서를 바탕으로, 실제 개발 착수와 외주 전달이 가능한 수준의 데이터 모델, SQL DDL, API 계약, PBX 설정 초안을 통합 정리한 문서이다. 핵심 원칙은 Asterisk는 통화 제어에 집중하고, CTI Middleware가 이벤트 정규화와 업무 로직을 담당하는 구조다.

구성 영역	핵심 산출물	실무 목적
데이터 모델	ERD, 관계 정의, 핵심 엔티티 설명	세션 정합성 및 운영 데이터 표준화
DB 스키마	PostgreSQL DDL, 인덱스, 제약조건	개발 초기 스키마 기준선 확보
API 계약	상담원/관리자/고객조회/콜제어 API	프론트엔드, 백엔드, 미들웨어 인터페이스 고정
PBX 설정	pjsip.conf, queues.conf, extensions 계열	트렁크, Queue, 녹취, 내선 흐름 초기 구동

문서 버전: v1.0

작성 기준일: 2026-04-12

목차

Placeholder for table of contents	0
-----------------------------------	---

1. 문서 적용 범위와 설계 기준

본 산출물은 Asterisk 22 LTS, PJSIP, Queue, MixMonitor, AMI 기반 CTI Middleware, PostgreSQL 업무 DB, 상담원 웹 CTI와 관리자 콘솔을 전제로 한다. 상세 설계서에서 정의한 계층 분리 원칙과 linkedid 중심 세션 모델을 데이터 구조와 API에 그대로 반영한다.

핵심 설계 원칙

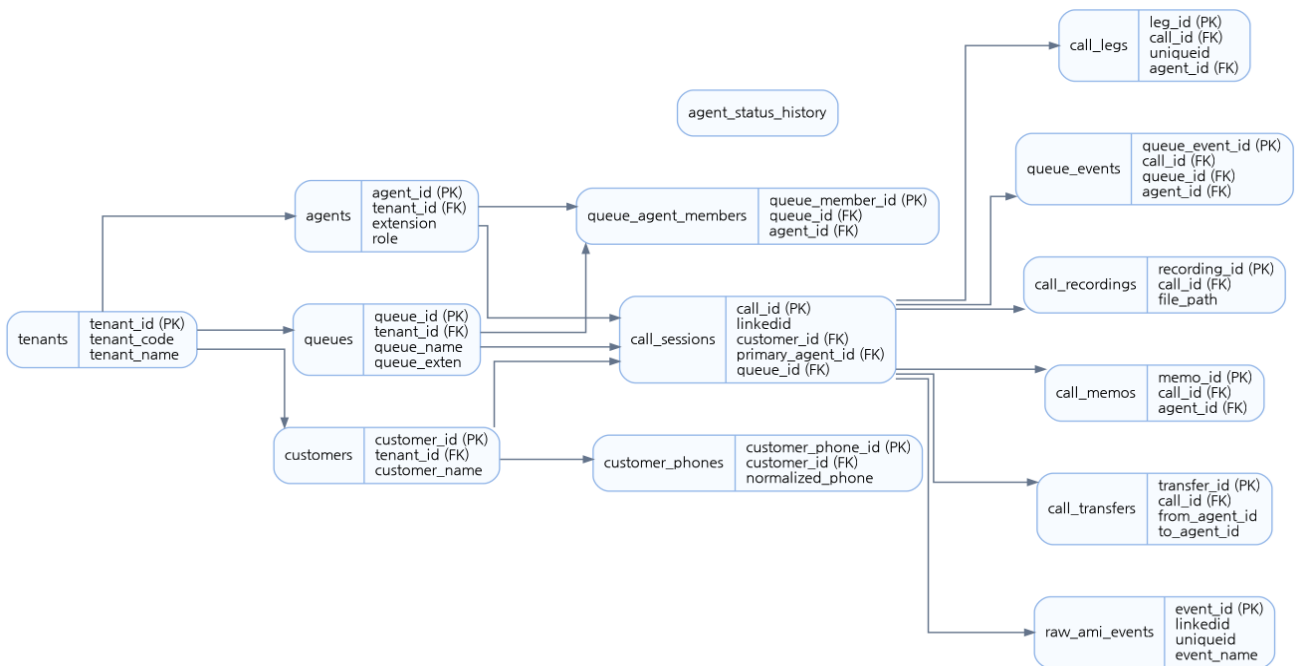
- Asterisk는 통화 제어, Queue, 녹취, SIP 단말 제어에 집중한다.
- CTI Middleware는 AMI 이벤트를 수집하여 call session 상태머신을 구성한다.
- 업무 데이터는 PostgreSQL에 저장하고 PBX 내부 상태를 UI가 직접 해석하지 않는다.
- linkedid를 비즈니스 세션의 기준 키로 삼고 uniqueid는 개별 leg 추적에 사용한다.
- 원본 이벤트(raw)와 정규화 결과(call_sessions, queue_events 등)를 분리 저장한다.

구현 범위 요약

영역	포함	비고
PBX	대표번호 인입, IVR, Queue, 내선, 녹취	Asterisk + PJSIP + app_queue + MixMonitor
미들웨어	AMI 수집, 상태머신, 고객 조회, REST/WebSocket	Node.js 또는 Python 가능
DB	상담원, 고객, 콜 세션, 레그, Queue 이벤트, 녹취, 메모	PostgreSQL 기준
UI	상담원 CTI, 관리자 대시보드	PBX 직접 접속 금지

2. DB ERD

ERD는 통화 세션 헤더인 call_sessions를 중심으로 call_legs, queue_events, call_recordings, call_memos, raw_ami_events가 연결되는 구조다. 상담원과 Queue는 다대다 관계이므로 queue_agent_members로 분리한다.



ERD 핵심 해석

- call_sessions는 linkedid 단위의 업무 세션 헤더다.
- call_legs는 uniqueid 기준의 실제 채널/브리지 레그를 저장한다.
- queue_events는 Queue 입장, 배정, 연결, 포기, 완료 이벤트의 팩트 테이블이다.
- raw_ami_events는 장애 분석과 리플레이용 원문 보관소다.
- customers와 customer_phones를 분리해 번호 이력 및 다중 번호를 허용한다.

핵심 관계 매트릭스

부모 엔티티	자식 엔티티	관계	실무 의미
tenants	agents / queues / customers	1:N	지사 또는 운영 단위 기준 분리
queues	queue_agent_members	1:N	Queue 소속 상담원 관리
agents	queue_agent_members	1:N	상담원 다중 Queue 참여
customers	customer_phones	1:N	고객 번호 이력 관리
call_sessions	call_legs	1:N	세션 내 다수 레그 추적
call_sessions	queue_events	1:N	Queue 상태 이벤트 누적
call_sessions	call_recordings	1:N	전환/분기 포함 녹취 관리

3. PostgreSQL 스키마 설계

아래 스키마는 UUID 기본키, timestamptz, CHECK 제약조건, 운영 인덱스를 포함한 초기 운영안이다. 개발 초기에는 이 구조를 마이그레이션 기준선으로 삼고, 통계 집계나 파티셔닝은 운영 볼륨에 따라 2차 단계에서 확장한다.

3.1 테이블 목록 요약

테이블	목적	핵심 키
tenants	센터/지사 단위 분리	tenant_id
agents	상담원 마스터	agent_id, extension
queues	Queue 정책 및 식별 정보	queue_id, queue_name
queue_agent_members	Queue-상담원 매핑	queue_member_id
customers	고객 마스터	customer_id
customer_phones	고객 전화번호 이력	customer_phone_id, normalized_phone
call_sessions	통화 세션 헤더	call_id, linkedid
call_legs	개별 leg 추적	leg_id, uniqueid
queue_events	Queue 이벤트 팩트	queue_event_id
call_recordings	녹취 메타데이터	recording_id
call_memos	상담 메모/후처리	memo_id
call_transfers	전환 이력	transfer_id

테이블	목적	핵심 키
raw_ami_events	AMI 원문 로그	event_id

3.2 권장 인덱스

- call_sessions(linkedid), call_sessions(started_at DESC), call_sessions(primary_agent_id, started_at DESC)
- customer_phones(normalized_phone)
- call_legs(uniqueid), call_legs(linkedid), call_legs(agent_id, started_at DESC)
- queue_events(queue_name, event_time DESC), queue_events(linkedid)
- raw_ami_events(linkedid, event_time DESC), payload GIN 인덱스

3.3 PostgreSQL DDL

```

CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE TABLE tenants (
tenant_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_code VARCHAR(32) NOT NULL UNIQUE,
tenant_name VARCHAR(128) NOT NULL,
timezone VARCHAR(64) NOT NULL DEFAULT 'Asia/Seoul',
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE agents (
agent_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(tenant_id),
login_id VARCHAR(64) NOT NULL,
login_password_hash VARCHAR(255) NOT NULL,
agent_code VARCHAR(32) NOT NULL,
agent_name VARCHAR(128) NOT NULL,
extension VARCHAR(16) NOT NULL,
sip_endpoint VARCHAR(64),
role VARCHAR(32) NOT NULL DEFAULT 'agent'
CHECK (role IN ('agent', 'supervisor', 'admin')),
employment_status VARCHAR(16) NOT NULL DEFAULT 'active'
CHECK (employment_status IN ('active', 'inactive', 'retired')),
default_queue_id UUID NULL,
last_login_at TIMESTAMPTZ NULL,
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
UNIQUE (tenant_id, login_id),
UNIQUE (tenant_id, agent_code),
UNIQUE (tenant_id, extension)
);

CREATE TABLE queues (
queue_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(tenant_id),
queue_name VARCHAR(64) NOT NULL,
queue_exten VARCHAR(16) NOT NULL,
queue_display_name VARCHAR(128) NOT NULL,
strategy VARCHAR(32) NOT NULL DEFAULT 'leastrecent',
ring_timeout_seconds INTEGER NOT NULL DEFAULT 15,
retry_seconds INTEGER NOT NULL DEFAULT 3,
wrapup_seconds INTEGER NOT NULL DEFAULT 30,
max_wait_seconds INTEGER NOT NULL DEFAULT 45,
autopause BOOLEAN NOT NULL DEFAULT TRUE,
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
UNIQUE (tenant_id, queue_name),
UNIQUE (tenant_id, queue_exten)
);

```

```

CREATE TABLE queue_agent_members (
queue_member_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(tenant_id),
queue_id UUID NOT NULL REFERENCES queues(queue_id) ON DELETE CASCADE,
agent_id UUID NOT NULL REFERENCES agents(agent_id) ON DELETE CASCADE,
penalty INTEGER NOT NULL DEFAULT 0,
member_order INTEGER NOT NULL DEFAULT 0,
is_active BOOLEAN NOT NULL DEFAULT TRUE,
joined_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
UNIQUE (queue_id, agent_id)
);

CREATE TABLE customers (
customer_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(tenant_id),
customer_code VARCHAR(32),
customer_name VARCHAR(128),
company_name VARCHAR(128),
grade VARCHAR(32),
status VARCHAR(32) DEFAULT 'active',
memo TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE customer_phones (
customer_phone_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
customer_id UUID NOT NULL REFERENCES customers(customer_id) ON DELETE CASCADE,
phone_number VARCHAR(32) NOT NULL,
normalized_phone VARCHAR(32) NOT NULL,
phone_type VARCHAR(16) NOT NULL DEFAULT 'mobile'
CHECK (phone_type IN ('mobile', 'home', 'office', 'other')),
is_primary BOOLEAN NOT NULL DEFAULT FALSE,
is_active BOOLEAN NOT NULL DEFAULT TRUE,
verified_at TIMESTAMPTZ NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_customer_phones_normalized_phone
ON customer_phones(normalized_phone);

CREATE TABLE agent_status_history (
status_history_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(tenant_id),
agent_id UUID NOT NULL REFERENCES agents(agent_id),
status_code VARCHAR(32) NOT NULL
CHECK (status_code IN (
'LOGGED_OUT', 'AVAILABLE', 'RINGING', 'TALKING',
'AFTER_CALL_WORK', 'BREAK', 'MEAL', 'TRAINING',
'SYSTEM_PAUSED', 'MANUAL_PAUSED'
)),
reason_code VARCHAR(32),
reason_text VARCHAR(255),
source_type VARCHAR(16) NOT NULL DEFAULT 'system'
CHECK (source_type IN ('system', 'agent', 'admin')),
started_at TIMESTAMPTZ NOT NULL,
ended_at TIMESTAMPTZ NULL,
duration_seconds INTEGER NOT NULL DEFAULT 0,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE call_sessions (
  call_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(tenant_id),
  linkedid VARCHAR(32) NOT NULL,
  direction VARCHAR(16) NOT NULL
  CHECK (direction IN ('inbound', 'outbound', 'internal')),
  ani VARCHAR(32),
  ani_normalized VARCHAR(32),
  dnis VARCHAR(32),
  did_number VARCHAR(32),
  trunk_name VARCHAR(64),
  queue_id UUID NULL REFERENCES queues(queue_id),
  queue_name VARCHAR(64),
  customer_id UUID NULL REFERENCES customers(customer_id),
  primary_agent_id UUID NULL REFERENCES agents(agent_id),
  session_status VARCHAR(32) NOT NULL
  CHECK (session_status IN (
    'NEW', 'IVR', 'QUEUED', 'RINGING_AGENT', 'TALKING',
    'HOLD', 'TRANSFERRING', 'AFTER_CALL_WORK', 'ENDED'
  )),
  campaign_code VARCHAR(64),
  result_code VARCHAR(32),
  result_detail VARCHAR(255),
  abandon_flag BOOLEAN NOT NULL DEFAULT FALSE,
  transfer_flag BOOLEAN NOT NULL DEFAULT FALSE,
  callback_flag BOOLEAN NOT NULL DEFAULT FALSE,
  recording_flag BOOLEAN NOT NULL DEFAULT FALSE,
  started_at TIMESTAMPTZ NOT NULL,
  queued_at TIMESTAMPTZ NULL,
  ringing_at TIMESTAMPTZ NULL,
  answered_at TIMESTAMPTZ NULL,
  ended_at TIMESTAMPTZ NULL,
  wait_seconds INTEGER NOT NULL DEFAULT 0,
  ring_seconds INTEGER NOT NULL DEFAULT 0,
  talk_seconds INTEGER NOT NULL DEFAULT 0,
  hold_seconds INTEGER NOT NULL DEFAULT 0,
  acw_seconds INTEGER NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (tenant_id, linkedid)
);

CREATE TABLE call_legs (
  leg_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  call_id UUID NOT NULL REFERENCES call_sessions(call_id) ON DELETE CASCADE,
  tenant_id UUID NOT NULL REFERENCES tenants(tenant_id),
  linkedid VARCHAR(32) NOT NULL,
  uniqueid VARCHAR(32) NOT NULL,
  parent_uniqueid VARCHAR(32),
  leg_type VARCHAR(16) NOT NULL
  CHECK (leg_type IN ('inbound', 'agent', 'outbound', 'transfer', 'ivr', 'local')),
  channel_name VARCHAR(128),
  bridged_channel_name VARCHAR(128),
  endpoint VARCHAR(64),
  agent_id UUID NULL REFERENCES agents(agent_id),
  state_code VARCHAR(32),
  dial_status VARCHAR(32),
  started_at TIMESTAMPTZ NOT NULL,
  bridged_at TIMESTAMPTZ NULL,
  ended_at TIMESTAMPTZ NULL,
  duration_seconds INTEGER NOT NULL DEFAULT 0,
  disposition VARCHAR(32),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (tenant_id, uniqueid)
);

```

```

CREATE TABLE queue_events (
  queue_event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  call_id UUID NULL REFERENCES call_sessions(call_id) ON DELETE CASCADE,
  tenant_id UUID NOT NULL REFERENCES tenants(tenant_id),
  linkedid VARCHAR(32) NOT NULL,
  queue_id UUID NULL REFERENCES queues(queue_id),
  queue_name VARCHAR(64) NOT NULL,
  agent_id UUID NULL REFERENCES agents(agent_id),
  event_type VARCHAR(32) NOT NULL
  CHECK (event_type IN (
    'JOIN', 'LEAVE', 'ABANDON', 'TIMEOUT',
    'AGENT_CALLED', 'AGENT_CONNECT', 'AGENT_COMPLETE'
  )),
  event_time TIMESTAMPTZ NOT NULL,
  wait_seconds INTEGER NOT NULL DEFAULT 0,
  ring_seconds INTEGER NOT NULL DEFAULT 0,
  position_no INTEGER,
  raw_payload JSONB,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE call_recordings (
  recording_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  call_id UUID NOT NULL REFERENCES call_sessions(call_id) ON DELETE CASCADE,
  tenant_id UUID NOT NULL REFERENCES tenants(tenant_id),
  linkedid VARCHAR(32) NOT NULL,
  recording_type VARCHAR(16) NOT NULL DEFAULT 'mixmonitor'
  CHECK (recording_type IN ('mixmonitor', 'monitor', 'external')),
  file_path TEXT NOT NULL,
  file_name VARCHAR(255) NOT NULL,
  file_format VARCHAR(16) NOT NULL DEFAULT 'wav',
  file_size_bytes BIGINT,
  storage_provider VARCHAR(32) DEFAULT 'local',
  recording_started_at TIMESTAMPTZ,
  recording_ended_at TIMESTAMPTZ,
  duration_seconds INTEGER NOT NULL DEFAULT 0,
  checksum_sha256 VARCHAR(64),
  is_access_restricted BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE call_memos (
  memo_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  call_id UUID NOT NULL REFERENCES call_sessions(call_id) ON DELETE CASCADE,
  tenant_id UUID NOT NULL REFERENCES tenants(tenant_id),
  agent_id UUID NOT NULL REFERENCES agents(agent_id),
  customer_id UUID NULL REFERENCES customers(customer_id),
  memo_type VARCHAR(16) NOT NULL DEFAULT 'acw'
  CHECK (memo_type IN ('live', 'acw', 'supervisor', 'system')),
  result_code VARCHAR(32),
  sub_result_code VARCHAR(32),
  memo_text TEXT,
  is_final BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE call_transfers (
  transfer_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  call_id UUID NOT NULL REFERENCES call_sessions(call_id) ON DELETE CASCADE,
  tenant_id UUID NOT NULL REFERENCES tenants(tenant_id),
  linkedid VARCHAR(32) NOT NULL,
  transfer_type VARCHAR(16) NOT NULL
  CHECK (transfer_type IN ('blind', 'attended', 'queue', 'external')),
  from_agent_id UUID NULL REFERENCES agents(agent_id),
  to_agent_id UUID NULL REFERENCES agents(agent_id),
  from_extension VARCHAR(16),
  to_extension VARCHAR(32),
  transfer_result VARCHAR(32),
  requested_at TIMESTAMPTZ NOT NULL,
  completed_at TIMESTAMPTZ NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE raw_ami_events (
  event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(tenant_id),
  linkedid VARCHAR(32),
  uniqueid VARCHAR(32),
  event_name VARCHAR(64) NOT NULL,
  event_time TIMESTAMPTZ NOT NULL,
  sequence_no BIGSERIAL,
  payload JSONB NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

4. API 명세서

API는 REST 기반 업무 CRUD와 WebSocket 실시간 이벤트를 분리한다. 콜 제어 요청은 PBX로 직접 전달하지 않고 미들웨어가 권한 확인과 세션 적합성 검사를 거쳐 AMI/ARI 또는 내부 서비스로 위임한다.

4.1 공통 규칙

항목	규칙
Base URL	/api/v1
인증	Authorization: Bearer
응답 형식	success, data, error 공통 구조
권한	agent / supervisor / admin 분리
실시간 이벤트	WebSocket /ws/agent, /ws/admin 사용

4.2 주요 REST API

Method	Path	설명
POST	/auth/login	상담원 로그인 및 JWT 발급
POST	/auth/logout	로그아웃 처리
GET	/me/session	현재 상담원 세션 정보 조회
POST	/agents/{agentId}/status	상담원 상태 변경
GET	/customers/search?phone=...	전화번호 기반 고객 조회
GET	/calls/active	현재 활성 콜 목록

Method	Path	설명
GET	/calls/{callId}	콜 상세 조회
POST	/calls/{callId}/memo	상담 메모 및 후처리 코드 저장
POST	/calls/originate	클릭투콜 발신
POST	/calls/{callId}/transfer	호 전환 요청
POST	/calls/{callId}/hangup	호 종료 요청
GET	/queues/summary	Queue 요약 및 실시간 상태
GET	/admin/dashboard	관리자 대시보드 집계

4.3 요청/응답 예시

```

POST /api/v1/auth/login
{
  "loginId": "agent1001",
  "password": "*****",
  "extension": "1001"
}

GET /api/v1/customers/search?phone=01012345678
{
  "success": true,
  "data": [
    {
      "customerId": "uuid",
      "customerName": "김고객",
      "grade": "VIP",
      "phones": [
        { "phoneNumber": "01012345678", "isPrimary": true }
      ],
      "recentCalls": 5
    }
  ],
  "error": null
}

POST /api/v1/calls/{callId}/transfer
{
  "transferType": "attended",
  "targetType": "extension",
  "target": "1002"
}

```

4.4 WebSocket 이벤트

이벤트	용도	핵심 payload
call.created	신규 세션 생성	callId, linkedid, ani, queueName, sessionStatus
call.updated	상태 변화 반영	sessionStatus, agentId, answeredAt
call.ended	종료 및 통계 반영	endedAt, talkSeconds
screenpop.customer	고객 팝업	customerId, customerName, grade
agent.status.changed	상담원 상태 변경	agentId, statusCode
queue.summary.updated	Queue 집계 갱신	waitingCount, availableAgents

5. Asterisk 설정 파일 초안

Asterisk 설정은 역할별 파일로 분리하고, inbound, ivr, queue, agent, outbound 컨텍스트를 분리한다. 업무 규칙은 dialplan에 과도하게 넣지 않고 미들웨어와 DB가 담당한다.

5.1 pjsip.conf

```
[global]
type=global
user_agent=CallCenterPBX

[transport-udp]
type=transport
protocol=udp
bind=0.0.0.0:5060

[trunk-provider-auth]
type=auth
auth_type=userpass
username=TRUNK_ID
password=TRUNK_PASSWORD

[trunk-provider-aor]
type=aor
contact=sip:TRUNK_IP_OR_DOMAIN:5060

[trunk-provider-identify]
type=identify
endpoint=trunk-provider
match=TRUNK_IP

[trunk-provider]
type=endpoint
transport=transport-udp
context=inbound-main
disallow=all
allow=alaw,ulaw
aors=trunk-provider-aor
outbound_auth=trunk-provider-auth
from_user=TRUNK_ID
from_domain=TRUNK_IP_OR_DOMAIN
direct_media=no
rtp_symmetric=yes
force_rport=yes
rewrite_contact=yes
trust_id_inbound=yes
send_pai=yes
```

5.2 extensions.conf

```
[globals]
COMPANY_MAIN_DID=07012345678
REC_BASE_DIR=/var/spool/asterisk/monitor
DEFAULT_QUEUE=sales

#include extensions_inbound.conf
#include extensions_queue.conf
#include extensions_agent.conf
```

5.3 extensions_inbound.conf

```
[inbound-main]
exten => 07012345678,1,NoOp(Inbound DID ${EXTEN} / CALLERID=${CALLERID(num)})
same => n,Set(__TENANT_CODE=main)
same => n,Set(__ENTRY_DID=${EXTEN})
same => n,Set(__CALL_LINKEDID=${CHANNEL(linkedid)})
same => n,Set(__ANI=${CALLERID(num)})
same => n,Set(__DNIS=${EXTEN})
same => n,Set(CHANNEL(language)=ko)
same => n,Goto(inbound-ivr,s,1)
```

```
[inbound-ivr]
exten => s,1,NoOp(Inbound IVR Start / LINKEDID=${CHANNEL(linkedid)})
same => n,Answer()
same => n,Playback(custom/welcome)
same => n,Background(custom/main_menu)
same => n,WaitExten(5)
```

```
exten => 1,1,NoOp(Route to Sales Queue)
same => n,Set(__QUEUE_NAME=sales)
same => n,Goto(queue-entry,s,1)
```

5.4 extensions_queue.conf

```
[queue-entry]
exten => s,1,NoOp(Queue Entry / QUEUE=${QUEUE_NAME} / LINKEDID=${CHANNEL(linkedid)})
same => n,Set(__REC_FILE=${STRFTIME(${EPOCH},,%Y/%m/%d)}/${CHANNEL(linkedid)}-${UNIQUEID}.wav)
same => n,Set(__CALL_START_TS=${STRFTIME(${EPOCH},,%Y-%m-%d %H:%M:%S)})
same => n,MixMonitor(${REC_BASE_DIR}/${REC_FILE},b)
same => n,Set(CDR(userfield)=linkedid=${CHANNEL(linkedid)};queue=${QUEUE_NAME};rec=${REC_FILE})
same => n,Queue(${QUEUE_NAME},tT,,45)
same => n,NoOp(Queue Result: ${QUEUESTATUS})
same => n,Goto(queue-exit,s,1)
```

```
[queue-exit]
exten => s,1,NoOp(Queue Exit / STATUS=${QUEUESTATUS} / ABANDONED?=${ABANDONED})
same => n,ExecIf("${QUEUESTATUS}"="TIMEOUT")?Playback(custom/queue_timeout)
same => n,Hangup()
```

5.5 queues.conf

```
[general]
persistentmembers=yes
autofill=yes
shared_lastcall=yes
keep_stats=yes

[sales]
musicclass=default
strategy=leastrecent
timeout=15
retry=3
wrapuptime=30
maxlen=0
autopause=yes
setinterfacevar=yes
setqueuevar=yes
setqueueentryvar=yes
ringinuse=no
member => Local/1001@from-queue/n,0,Agent1001
member => Local/1002@from-queue/n,0,Agent1002
```

5.6 manager.conf

```
[general]
enabled = yes
webenabled = no
port = 5038
bindaddr = 127.0.0.1

[cti_middleware]
secret = STRONG_AMI_PASSWORD
deny = 0.0.0.0/0.0.0.0
permit = 127.0.0.1/255.255.255.255
read = system,call,agent,user,dtmf,log,verbose,command,reporting,cd,rdialplan
write = system,call,agent,user,command,originate,message
writetimeout = 5000
```

6. 이벤트 처리 매핑 요약

AMI 이벤트	DB 반영	세션 상태 변화
Newchannel	call_legs 생성	NEW 유지
QueueCallerJoin	queue_events(JOIN), call_sessions.queued_at	QUEUED
AgentCalled	queue_events(AGENT_CALLED)	RINGING_AGENT
AgentConnect / BridgeEnter	answered_at, primary_agent_id	TALKING
BridgeLeave / Hold	hold_seconds 누적	HOLD 또는 TALKING
Hangup	leg 종료, 종료시간 기록	ENDED 후보
AgentComplete	queue_events(AGENT_COMPLETE), acw 시작	AFTER_CALL_WORK

7. 구현 우선순위

- 1단계: pjsip, inbound, Queue, MixMonitor, manager.conf 기반 PBX PoC 구동
- 2단계: AMI Connector와 raw_ami_events 적재
- 3단계: Session Engine으로 call_sessions / call_legs / queue_events 정규화
- 4단계: 고객 조회 API, screen pop, 상담원 상태 관리
- 5단계: 메모, 결과코드, 전환, 관리자 대시보드

결론적으로 본 문서는 개발팀이 바로 마이그레이션 스크립트를 생성하고, 백엔드 API와 PBX 설정 초안을 병행 구현할 수 있도록 하는 초기 기준선이다. 실제 운영 전에는 통신사 SIP trunk 상세 스펙, DID 라우팅 규칙, 녹취 파일 스토리지 정책, 개인정보 마스킹 정책을 추가 확정해야 한다.